

CS 546

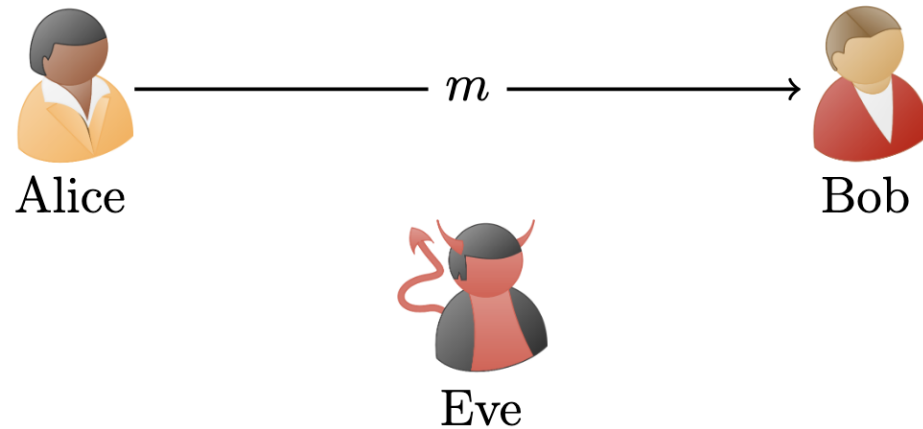
Computer System Security

Lecture 2: Crypto (Confidentiality - II)

Instructor: Aarushi Goel

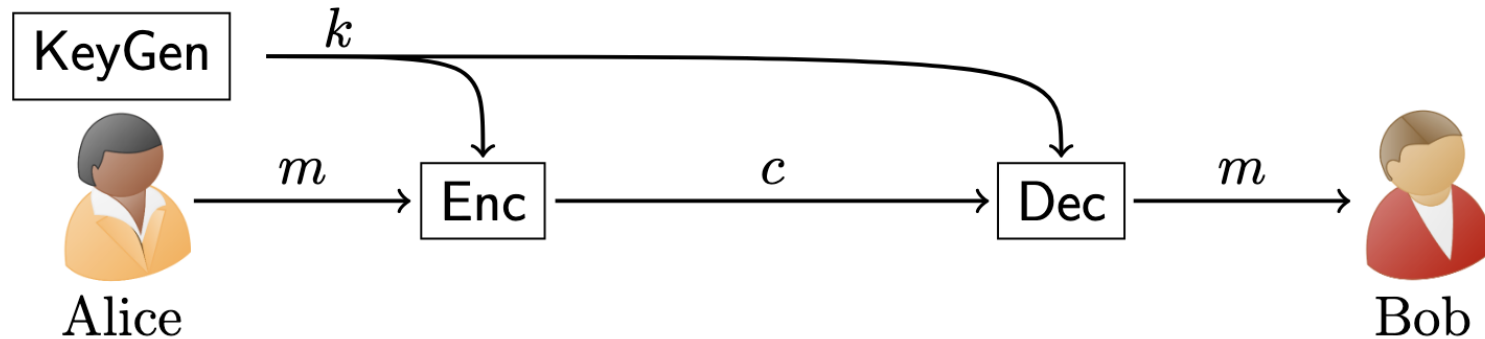
Fall 2026

Problem Setting: Secret Communication



- Alice wants to send a message m to Bob.
- How can Alice convey this message to Bob, while keeping it hidden from an eavesdropper Eve?
- In other words, how can we ensure confidentiality of m ?

Problem Setting: Secret Communication



- **Key Generation:** KeyGen is the randomized process of choosing a secret key k , which is used as input to both Enc and Dec algorithms.
- **Encryption:** Alice transforms (encrypts) the message m – also called the “plaintext” – into a “ciphertext” c .
- **Communication:** Alice sends c to Bob.
- **Decryption:** Bob recovers (decrypts) the original m from c .

Kerckhoffs' Principle

“The cipher method must not be required to be secret, and it must be able to fall into the hands of the enemy without inconvenience.”

- If the algorithms themselves are not secret, then there must be some other secret information in the system.
- That information is called the **secret key**.
- If a secret key gets compromised, we only need to choose a new one, not reinvent a new encryption algorithm.
- Put differently, the security of the system is concentrated in the secrecy of the key, not the algorithm.



Auguste Kerckhoffs
19th-century
Dutch cryptographer

Classical Ciphers/Encryption Mechanisms

(Vigenère: Poly-alphabetic Shift Cipher (1300s))

KeyGen: Key k is a random word of length t .

Encryption Algorithm: as an example, say $k = CRYPTO$

msg

W	H	A	T	A	N	I	C	E	D	A	Y	T	O	D	A	Y
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

+ mod 26

key

C	R	Y	P	T	O	C	R	Y	P	T	O	C	R	Y	P	T
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

cipher

Y	Y	Y	I	T	B	K	T	C	S	T	M	V	F	B	P	R
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Exercise

Think of an attack!

This cipher was a lot harder to break. In fact, a systematic attack took years to devise.

Classical Ciphers/Encryption Mechanisms

(How to break Vigenère cipher)

Warm up Case: Let us assume that we already know that the size of key k is 6.

msg

W	H	A	T	A	N	I	C	E	D	A	Y	T	O	D	A	Y
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

key

C	R	Y	P	T	O	C	R	Y	P	T	O	C	R	Y	P	T
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

+ mod 26

cipher

Y	Y	Y	I	T	B	K	T	C	S	T	M	V	F	B	P	R
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Naïve Approach: Every 6th character is encrypted under the same alphabet. We can do a separate frequency analysis on “YKV..”, “YTF..”, and so on.

- The problem is that this approach still requires some guesswork.

Classical Ciphers/Encryption Mechanisms

(How to break Vigenère cipher)

Warm up Case: Let us assume that we already know that the size of key k is 6.

msg

W	H	A	T	A	N	I	C	E	D	A	Y	T	O	D	A	Y
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

+ mod 26

key

C	R	Y	P	T	O	C	R	Y	P	T	O	C	R	Y	P	T
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

cipher

Y	Y	Y	I	T	B	K	T	C	S	T	M	V	F	B	P	R
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

A Better Approach:

- Let $p_i (0 \leq i \leq 25)$ denote the frequency of the i^{th} English alphabet in a normal English text.
 - One can compute $\sum_i p_i^2 \approx 0.065$.
- Let $q_i (0 \leq i \leq 25)$ denote the frequency of the i^{th} English alphabet in above text "YKV..".
 - If this set of alphabets is encrypted under the k^{th} alphabet, then $q_{i+k} \approx p_i$ for every i .
- To find k , search through $\{0, \dots, 25\}$ and output the k for which $\sum_i p_i q_{i+k}$ is closest to 0.065.
 - For a wrong k , this value will be closer to $\sum_i \left(\frac{1}{26}\right)^2 \approx 0.038$.

Classical Ciphers/Encryption Mechanisms

(How to break Vigenère cipher)

msg W H A T A N I C E D A Y T O D A Y

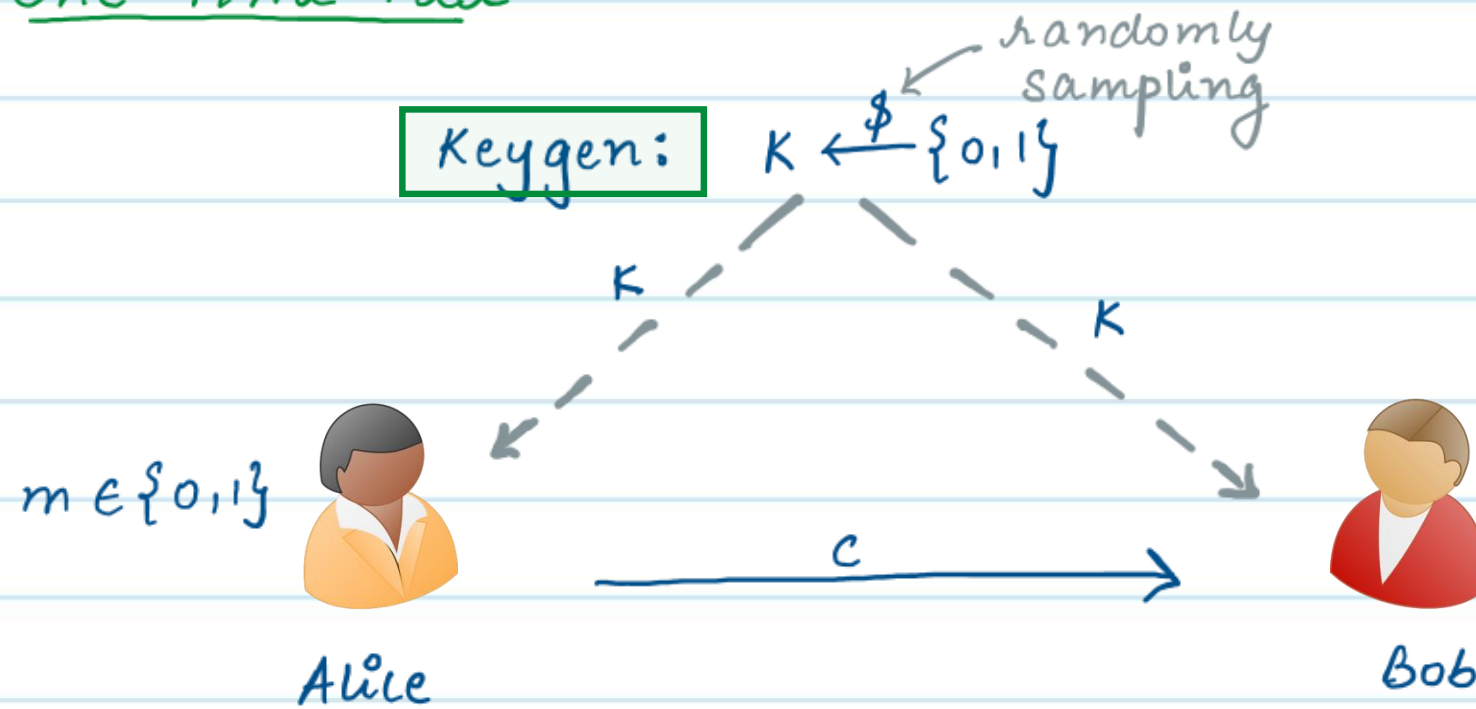
key C R Y P T O C R Y P T O C R Y P T

+ mod 26

cipher Y Y Y I T B K T C S T M V F B P R

When the size of key is unknown: Apply the same attack separately for all sizes up to some upper bound t .

One-Time Pad



$$\text{Enc}(m, K): c = m \oplus K$$

$$\text{Dec}(c, K): m = c \oplus K$$



Eve

(computationally
unbounded)

One-Time Pad

Let $m=0$. What are the possible values of c ?

prob	K	$c = m \oplus K$
$\frac{1}{2}$	0	0
$\frac{1}{2}$	1	1

$\Rightarrow$ whatever Eve sees
i.e., c is independent of m

also called the "view"
of the adversary

Perfect Secrecy

* Is the message m really secret?

* Eve could have easily guessed m with probability $\frac{1}{2}$

In fact if they already knew something about m , they can do even better.

NOTE: we did not claim that m is random

* But Eve could have learnt this without looking at c .

This means c does not leak any additional information about m .

Perfect Secrecy

Typical goal in Cryptography:

PRESERVE SECRECY !!

Intuitively speaking, this is what we want:

What Eve learns about m after seeing c , is the same as what they already knew about m .

Formalizing Perfect Secrecy

Setting 1: What did Eve already know about the message?

→ probability distribution over m

→ i.e., $\forall m, \Pr[\text{msg} = m]$

Setting 2: What does Eve learn after seeing c ?

→ New distribution $\Pr[\text{msg} = m \mid \text{view} = c]$

What do we want for secrecy?

Eve's knowledge in setting 1
=

Eve's knowledge in setting 2

Formalizing Perfect Secrecy

$$\Rightarrow \forall m, \forall c, \Pr[\text{msg}=m \mid \text{view}=c] = \Pr[\text{msg}=m]$$

view is independent of msg

$$\Rightarrow \forall m, \forall c, \Pr[\text{view}=c \mid \text{msg}=m] = \Pr[\text{view}=c]$$

for all possible values of the msg, the view is identically distributed.

$$\Rightarrow \forall c, \forall m_1, m_2, \Pr[\text{view}=c \mid \text{msg}=m_1] = \Pr[\text{view}=c \mid \text{msg}=m_2]$$

Formalizing Perfect Secrecy (Summary)

These are equivalent formulations:

$$\Rightarrow \forall m, \forall c, \Pr[\text{msg} = m \mid \text{view} = c] = \Pr[\text{msg} = m]$$

$$\Rightarrow \forall m, \forall c, \Pr[\text{view} = c \mid \text{msg} = m] = \Pr[\text{view} = c]$$

$$\Rightarrow \forall m, \forall c, \Pr[\text{msg} = m, \text{view} = c] = \Pr[\text{msg} = m] \times \Pr[\text{view} = c]$$

$$\Rightarrow \forall c, \forall m_1, m_2, \Pr[\text{view} = c \mid \text{msg} = m_1] = \Pr[\text{view} = c \mid \text{msg} = m_2]$$

To prove that an encryption is perfectly secure, it suffices to prove any of these

One-time Pad Encryption has perfect Secrecy

To Prove: $\forall m \in \{0,1\}^n, ct \in \{0,1\}^n$, show that

$$\Pr[M=m | C=ct] = \Pr[M=m]$$

Proof: $\Pr[M=m | C=ct] = \frac{\Pr[(M=m) \cap (C=ct)]}{\Pr[C=ct]}$

$$= \frac{\Pr[C=ct | M=m]}{\Pr[C=ct]} \times \Pr[M=m]$$

← (arrow from $\Pr[C=ct | M=m]$ to the left box)
→ (arrow from $\Pr[C=ct]$ to the right box)

$\forall m \in \{0,1\}^n$

$$\begin{aligned}\Pr[C=ct | M=m] &= \Pr[\text{Enc}(m, K) = ct] \\ &= \Pr[(m \oplus K) = ct] \\ &= \Pr[K = ct \oplus m] \\ &= \frac{1}{2}\end{aligned}$$

$$\begin{aligned}&= \sum_{m' \in \{0,1\}^n} \Pr[C=ct | M=m'] \times \Pr[M=m'] \\ &= \frac{1}{2} \times \sum_{m' \in \{0,1\}^n} \Pr[M=m'] = \frac{1}{2}\end{aligned}$$

Reflection #1

Is $\Pr[\text{msg} = m_1 \mid \text{view} = v] = \Pr[\text{msg} = m_2 \mid \text{view} = v]$?

why / why not?

This is only true if the msg is uniformly distributed.

⇒ a good encryption scheme should be perfectly secret for any distribution of messages in the message space.

Reflection #2

In this construction of one-time pad encryption scheme, can we use the same key to encrypt two different msgs?

Why / why Not?

Let $c_1 = m_1 \oplus K$ and $c_2 = m_2 \oplus K$

if Eve sees c_1, c_2 , it can compute

$$\underline{m_1 \oplus m_2} = c_1 \oplus c_2$$

↑

this leaks more information about m_1, m_2 than Eve had before seeing c_1, c_2 .

⇒ This is an example of a "one-time" encryption.

Reflection #3

Is it sufficient for an encryption scheme to only have perfect secrecy?

If so, is the following a good encryption?

$\text{Enc}(K, m) = 0$, i.e., the encryption of any message m , using any key K is 0?

No, because Bob cannot correctly recover the original message m with certainty.

⇒ An encryption scheme should have both correctness and privacy.

Perfectly Secure Encryption

Definition: A perfectly secure encryption scheme with message space M , key space K , ciphertext space C , comprises of the following algorithms:

- * $\text{KeyGen} \rightarrow K$: This algorithm samples a key $K \in K$.
- * $\text{Enc}(K, m) \rightarrow c$: On input a key $K \in K$ and a message $m \in M$, it outputs ciphertext $c \in C$.
- * $\text{Dec}(K, c) \rightarrow m$: On input a key $K \in K$ and a ciphertext $c \in C$, it outputs message $m \in M$.

These algorithms must satisfy the following:

→ Correctness: $\forall K \in K, \forall m \in M$, it holds that:

$$\Pr[\text{Dec}(K, \text{Enc}(K, m)) = m] = 1$$

→ Perfect Secrecy: If for every probability distribution over M , $\forall m \in M$, and $\forall c \in C$ for which $\Pr[C=c] > 0$, it holds that:

$$\Pr[M=m | C=c] = \Pr[M=m]$$

Example of Insecure Encryption

Q Show that the following is not a perfectly secure encryption scheme.

* KeyGen: $K \xleftarrow{\$} \{0,1\}^n$

* Enc($K, m \in \{0,1\}^n$): $C = m \wedge K$ ↗ bit-wise AND

A 1) For some keys, there is no way to uniquely recover m .

counter example: if $K = 0^n$, then $C = m \wedge 0^n = 0^n$ for all $m \in \{0,1\}^n$.

⇒ correctness does not hold.

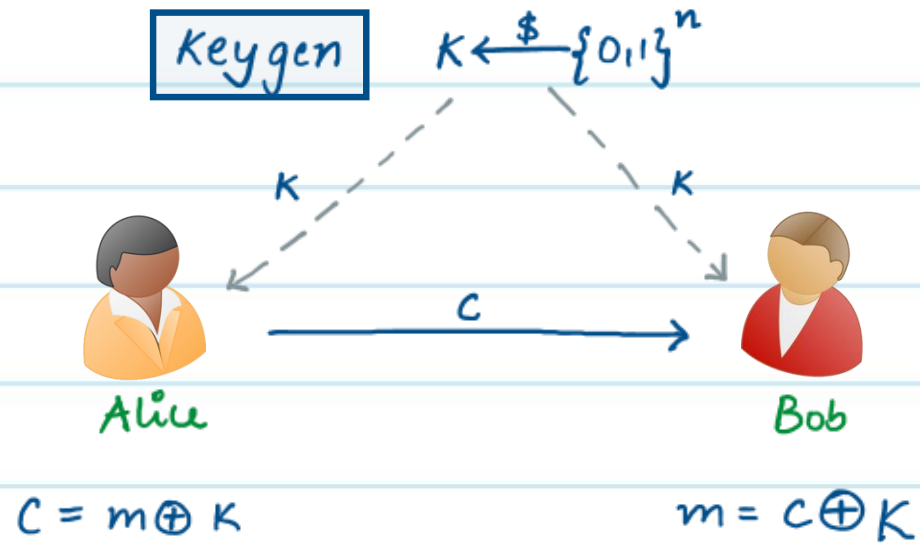
2) Also does not achieve perfect secrecy

We will show that $\exists m \in \{0,1\}^n$, such that $\Pr[C=c | M=m] \neq \Pr[C=c]$

counter example: Let $m = 0^n$

$$\Pr[C=0^n | m=0^n] = 1 \neq \Pr[C=c]$$

One-Time Pad Encryption



try to
formally
prove this!

Nice Feature:

- * Perfectly secure, i.e., secure against all types of Eve, not just PPT eves

Limitations:

- * Key must be as long as the message
- * Each key can only be used once.

Relaxing Security Requirements

What if the ciphertext is not exactly independent of the message?

Next Best Thing:

The distribution of ciphertexts is close to a distribution that is independent of the message.

Statistical Closeness
Computational Closeness



Computational Security vs Perfect Security

- * For perfect secrecy, we want security against *every* Eve/Adversary.
- * However, this may be an overkill.
- * As we discussed earlier, there are also limitations of perfectly secure encryption schemes.
- * In practice, it might be sufficient to security against computationally feasible attacks as opposed to all possible attacks



John Nash

“ It doesn't really matter whether attacks are impossible, only whether attacks are computationally infeasible ”

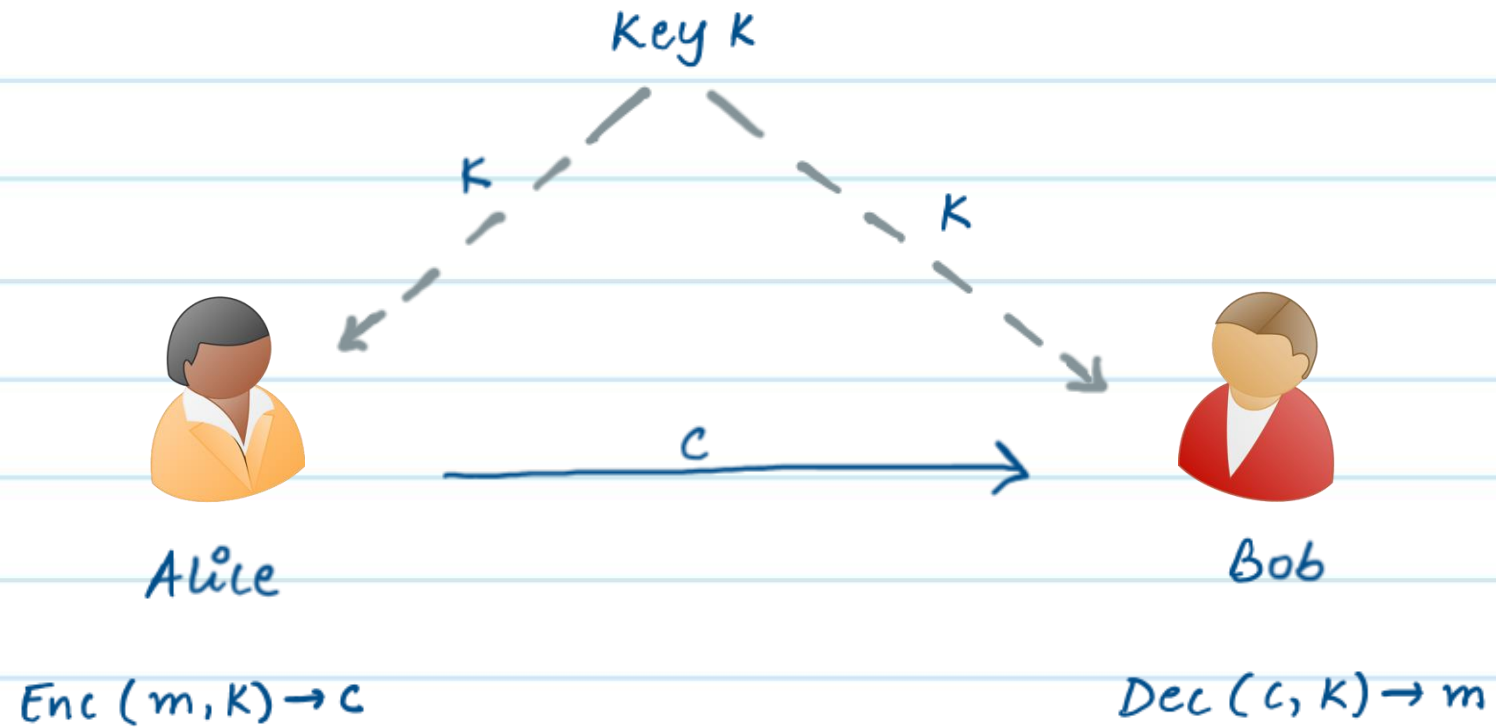
* Modern cryptography is based on this principle *

Cost of Computation

It can be helpful to think of cost of computation or the cost of attack in terms of monetary value. Following costs are approximated using the pricing model of Amazon EC2

<i>clock cycles</i>	<i>approx cost</i>	<i>reference</i>
2^{50}	\$3.50	<i>cup of coffee</i>
2^{55}	\$100	<i>decent tickets to a Portland Trailblazers game</i>
2^{65}	\$130,000	<i>median home price in Oshkosh, WI</i>
2^{75}	\$130 million	<i>budget of one of the Harry Potter movies</i>
2^{85}	\$140 billion	<i>GDP of Hungary</i>
2^{92}	\$20 trillion	<i>GDP of the United States</i>
2^{99}	\$2 quadrillion	<i>all of human economic activity since 300,000 BC⁴</i>
2^{128}	really a lot	<i>a billion human civilizations' worth of effort</i>

Computational Security



Computationally bounded
(polynomial-time algorithm)

Computational Assumptions

- * **Polynomial-time algorithm:** $A(x)$ with input x of length n is said to be polynomial time if A 's running time is $O(n^c)$ ^{some constant}

Randomized poly-time algorithms are called probabilistic-polynomial time (PPT) algorithms.

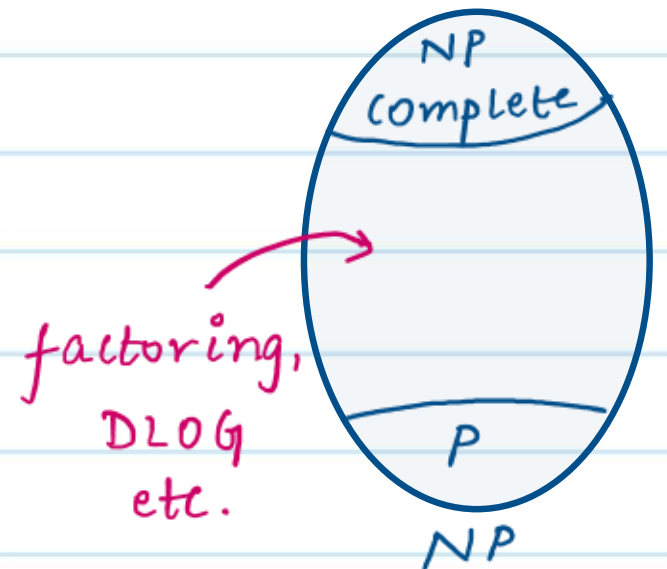
- * **NP-Problems:** Decision problems whose solution can be verified in poly time.

Example: graph isomorphism, graph 3-coloring

- * **NP-complete Problems:** "hardest" problems in NP.

Is $P = NP$?

we assume $P \neq NP$.



Factoring Assumption

Randomly sample $\rightarrow$ Miller-Rabin primality test.
How?

Generate two n -bit primes p, q ($p \neq q$)

Compute $N = p \cdot q$

Assumption: given N , it is computationally hard to find p & q (classically)

tunable parameter

* Such problems are assumed to be computationally hard.

* In other words, *we don't know* of any PPT algorithms to solve them

Note that this does not mean that PPT algorithms for solving them cannot exist. It simply means that no one has been able to find one yet.

This is why they are assumed to be hard

* Modern cryptographic primitives are based on the hardness assumptions of such problems. In particular, if a PPT Eve can solve these problems in poly/PPT time, then he can break security of the cryptographic scheme.

* When we talk about poly-time / PPT algorithms, what is the polynomial in? $\rightarrow$ we want polynomial in the problem size.

Computationally Secure Encryption.

An encryption scheme $(\text{KeyGen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is computationally secure if it satisfies **correctness** (as defined previously) and if for every $m_1, m_2 \in \mathcal{M}$,

$$\text{Enc}(K, m_1) \approx_c \text{Enc}(K, m_2)$$

←
Computationally indistinguishable:

- * Hard to distinguish for a computationally bounded adversary.
- * In other words, even if the adversary knows both m_1 & m_2 encryptions of m_1 & m_2 look similar to them if they are computationally bounded.

Candidate Construction for computationally Secure Encryption.

- * Recall the construction of one-time pad encryption

$$K \oplus m = C \quad \rightarrow \text{but this Key must be as long as the message.}$$

- * Potential Idea: $K \xrightarrow{G} G(K)$
 key some expansion function.

$$G(K) \oplus m = C$$

- * What is G ? Can it be something like $K \xrightarrow{G} K \parallel K \parallel K \parallel \dots$?
No! Remember Vignère cipher.

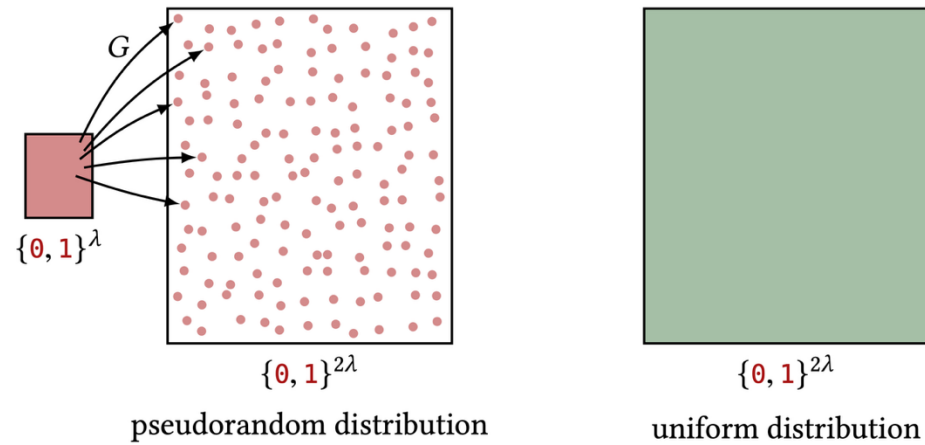
- * G should be a pseudorandom generator!

Pseudorandom Generators (PRG)

- * $G : \{0,1\}^n \rightarrow \{0,1\}^{\ell(n)}$, $\ell(n) > n$. PRGs are length expanding.
- * PRGs are deterministic functions
- * The output of a PRG is pseudorandom, i.e., it looks like a randomly sampled string to a computationally bounded adversary.

$$\{r \mid r \xleftarrow{\$} \{0,1\}^{\ell(n)}\} \approx_c \{G(s) \mid s \xleftarrow{\$} \{0,1\}^n\}$$

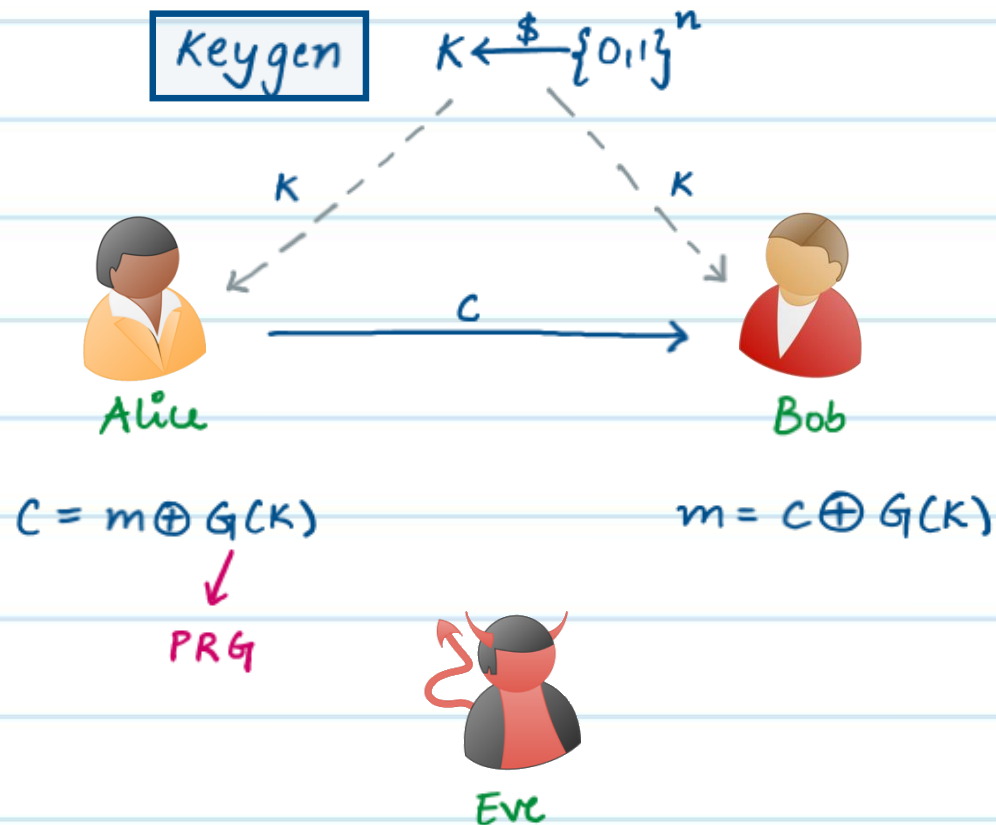
Illustrating PRG $G: \{0,1\}^n \rightarrow \{0,1\}^{2n}$



- * From a relative perspective, the PRG's output distribution is tiny. Out of the 2^{2n} possible strings in $\{0,1\}^{2n}$, only 2^n are possible outputs of G . These strings make up $2^n/2^{2n} = 1/2^n$ fraction of $\{0,1\}^{2n}$ — a negligible fraction.
- * From an absolute perspective, the PRG's output distribution is huge. There are 2^n possible outputs of G , which is an exponential amount. This is large enough that a PPT adversary cannot distinguish it from the set $\{0,1\}^{2n}$.

Pseudorandom OTP Encryption Scheme

- * Recall the candidate computationally secure encryption scheme from last class.



We want to show that this is indeed a computationally secure encryption

- * **correctness**: easy to verify
- * **security**: we need to prove that $\forall m_1, m_2$, the following distributions are computationally indistinguishable:
 $\{C = m_1 \oplus G(K) ; K \leftarrow \$_{\{0,1\}^n}\}$
 $\{C = m_2 \oplus G(K) ; K \leftarrow \$_{\{0,1\}^n}\}$

By definition of PRG:

$$\{G(K) \mid K \xleftarrow{\$} \{0,1\}^n\} \approx_c \{s \mid s \xleftarrow{\$} \{0,1\}^{\ell(n)}\}$$



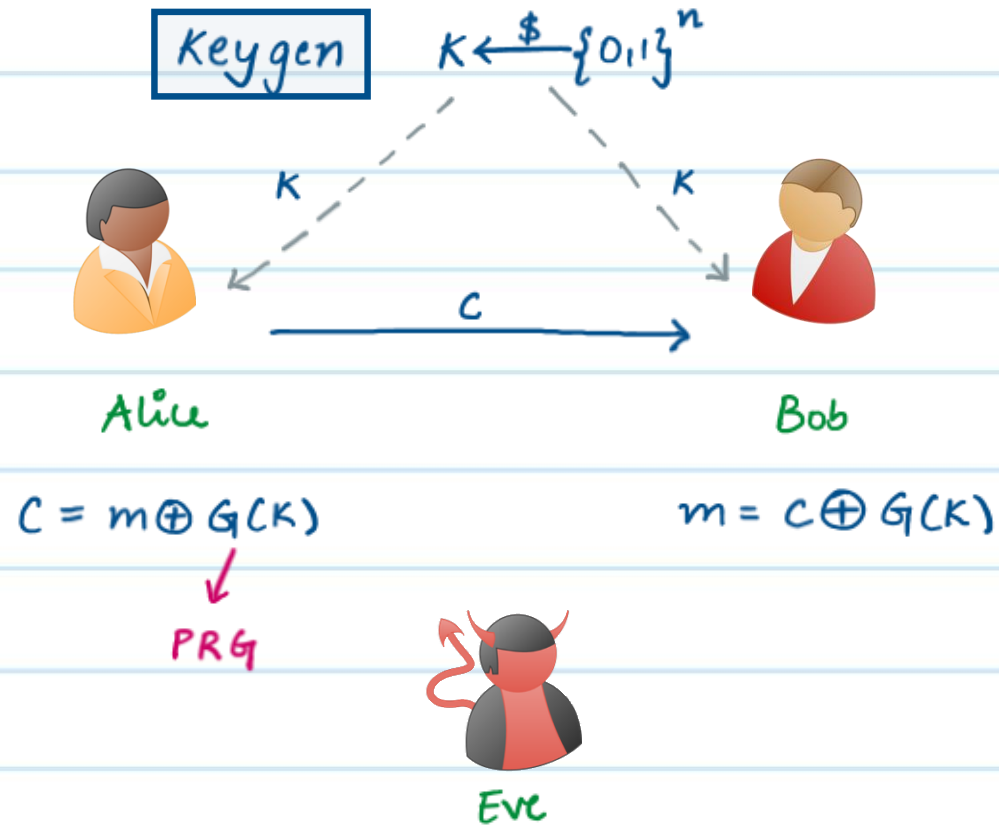
$$\{c = m_1 \oplus G(K) ; K \xleftarrow{\$} \{0,1\}^n\} \approx_c \{c = m_1 \oplus s ; s \xleftarrow{\$} \{0,1\}^{\ell(n)}\}$$

$$\approx_c \{c = m_2 \oplus s ; s \xleftarrow{\$} \{0,1\}^{\ell(n)}\}$$

$$\approx_c \{c = m_2 \oplus G(K) ; K \xleftarrow{\$} \{0,1\}^n\}$$

Pseudorandom OTP Encryption Scheme

* Recall the computationally secure encryption scheme based on PRGs.



Nice Feature:

* Short Key

Limitations:

* Each key can only be used once.

Why?

Pseudorandom Functions

- * Pseudorandom functions (PRFs) are Keyed functions of the form:

$$G: \underbrace{\{0,1\}^n}_{\text{Key}} \times \underbrace{\{0,1\}^n}_{\text{input}} \rightarrow \underbrace{\{0,1\}^n}_{\text{output}}$$

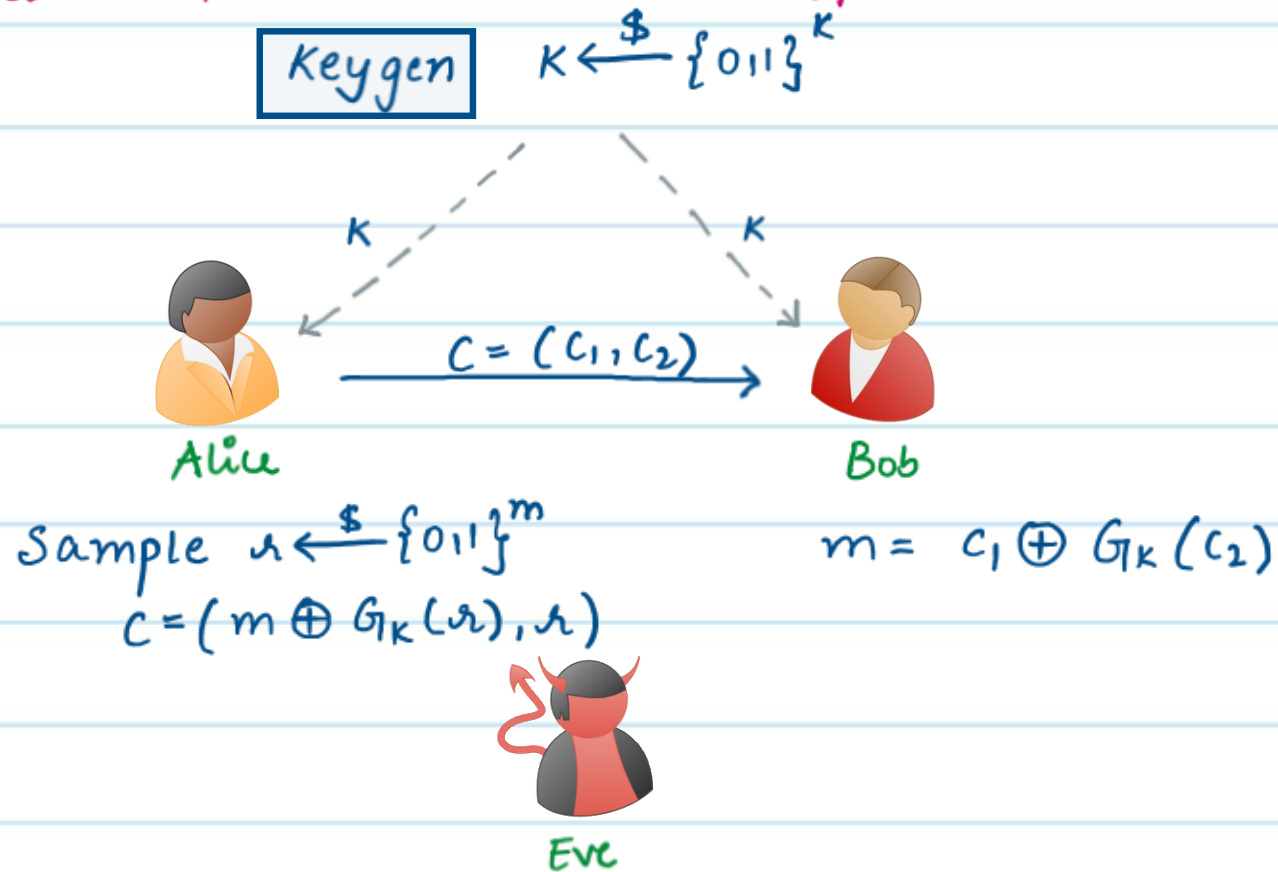
eg. $G_K(x) = y$

- * For any given key K , $G_K(x) \forall x \in \{0,1\}^n$ look computationally indistinguishable from random strings.

Encryption using PRFs

* Consider the following encryption scheme.

Such computationally secure encryption where the key can be re-used are also called CPA-secure encryption.



How to Build PRFs?

Two Approaches

Theoretical

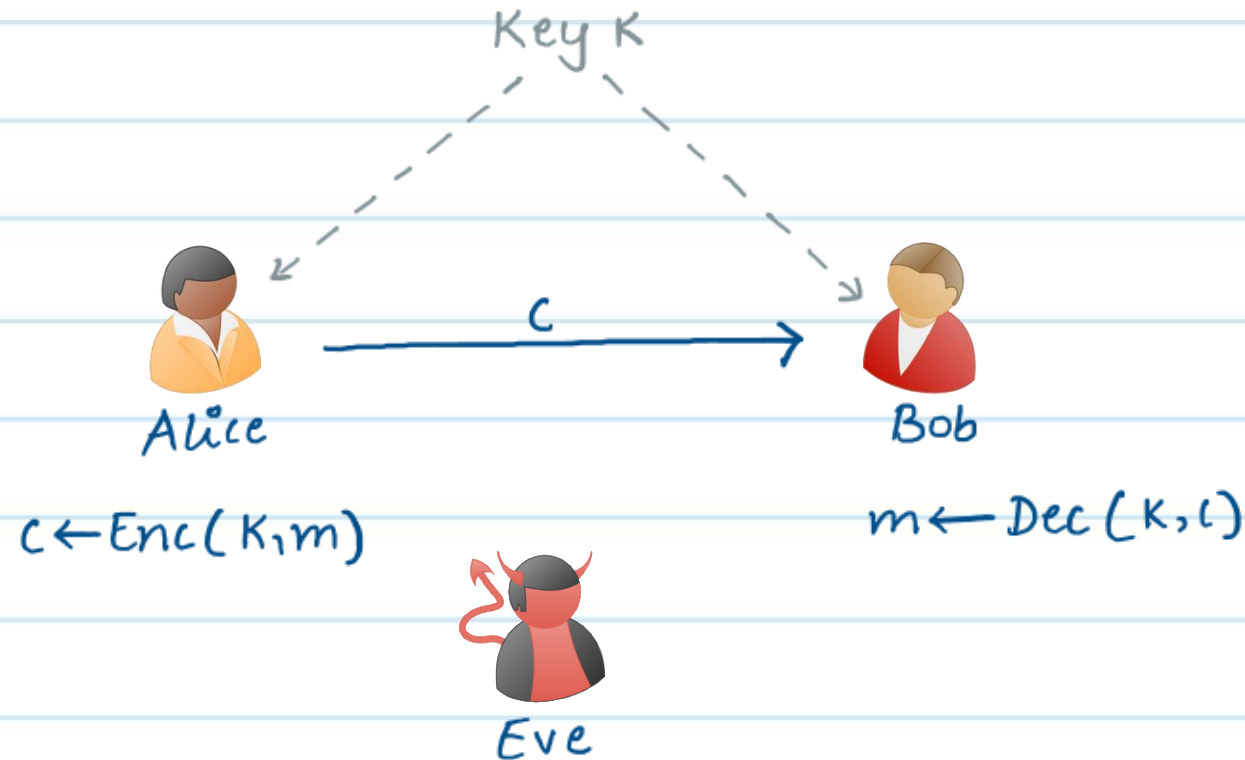
- * $PRG \Rightarrow PRF$
- * PRGs can be constructed using well-studied computational hardness assumptions.
- * Provably secure but far from practical.

Practical Constructions

- * Start from some design framework ("Appropriately chosen" functions, composed "appropriately many times" look random)
- * Build a candidate construction
- * Do extensive cryptanalysis.
- * Practical but not provably secure (eg. AES, DES etc.)

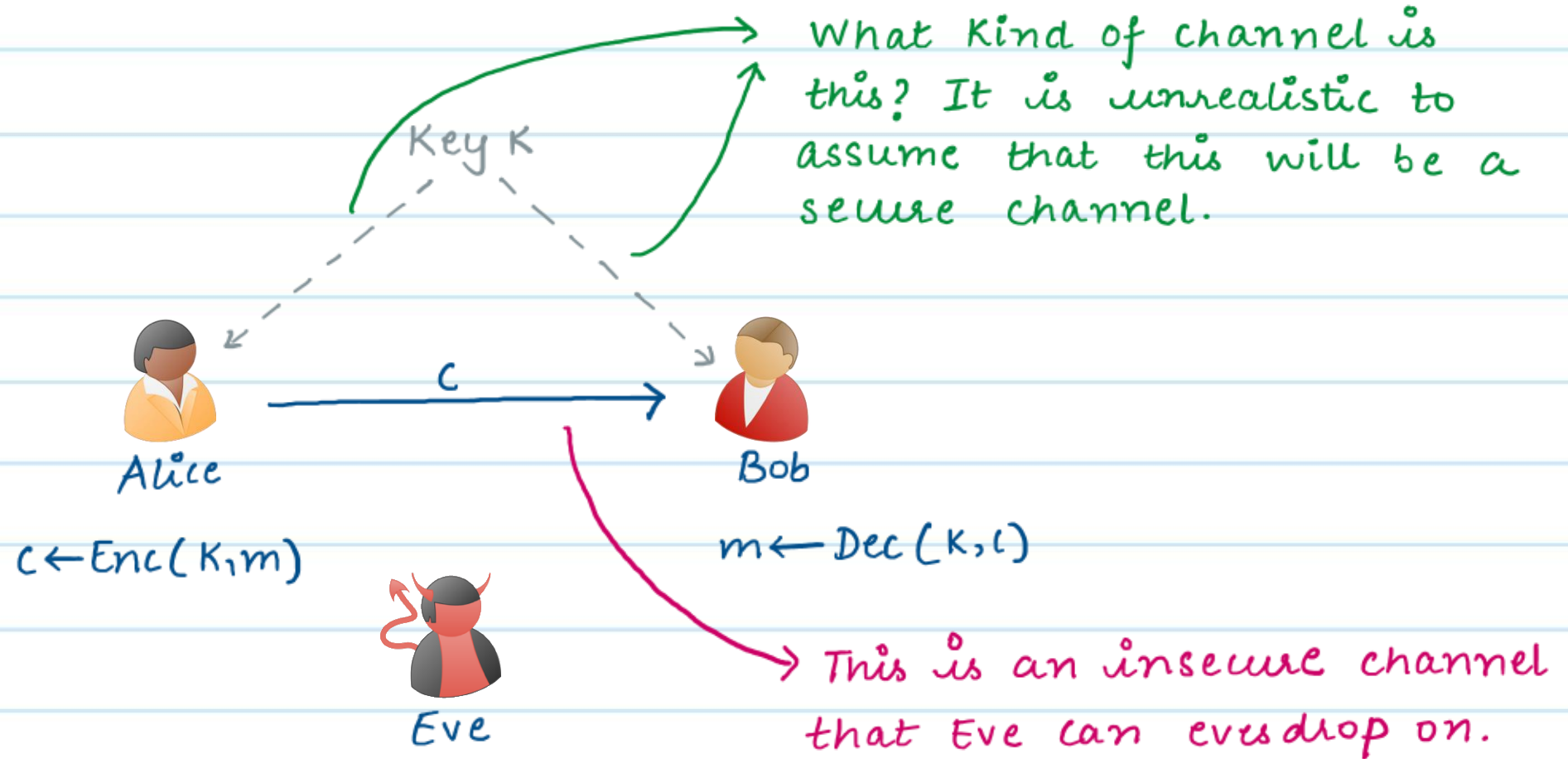
Encryption

- * So far, when discussing encryption schemes, we assumed that Alice and Bob have a pre-shared *secret* Key.



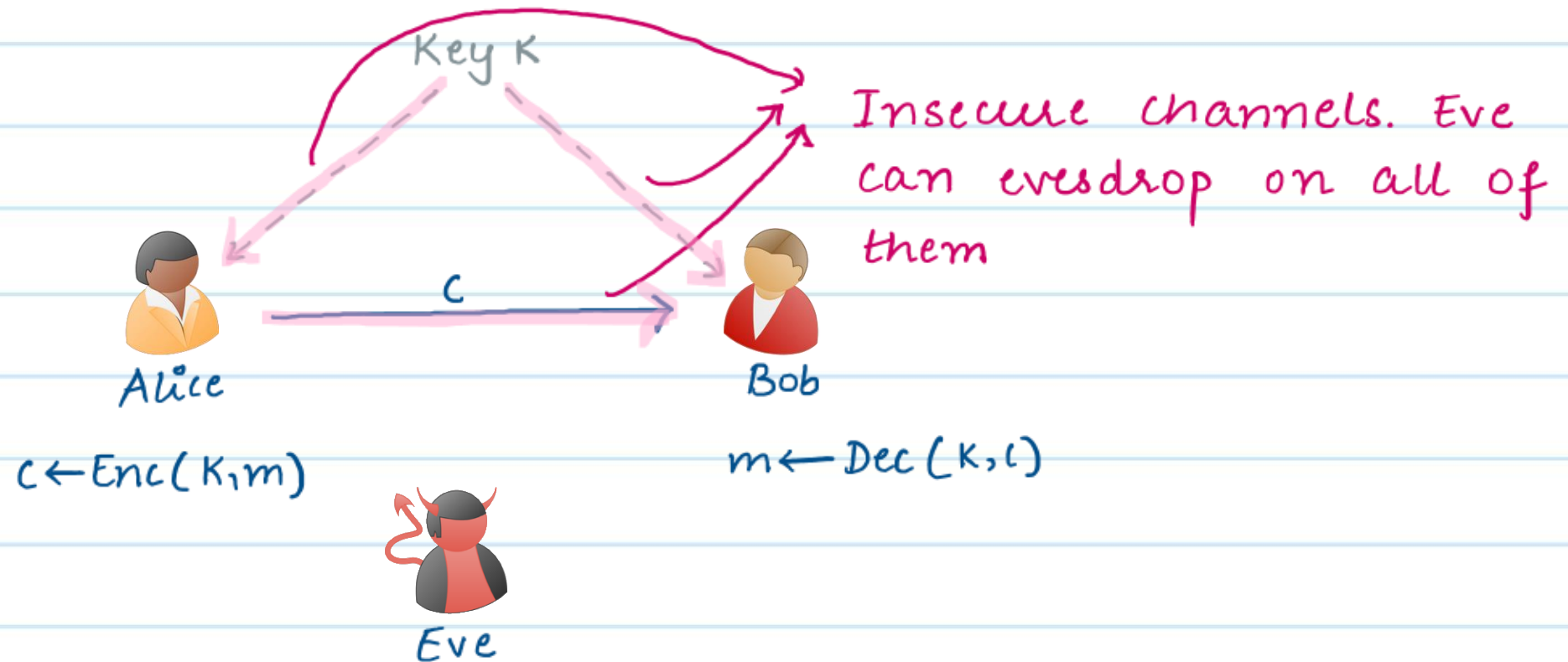
Key-Exchange Problem

* How do Alice and Bob share this key?



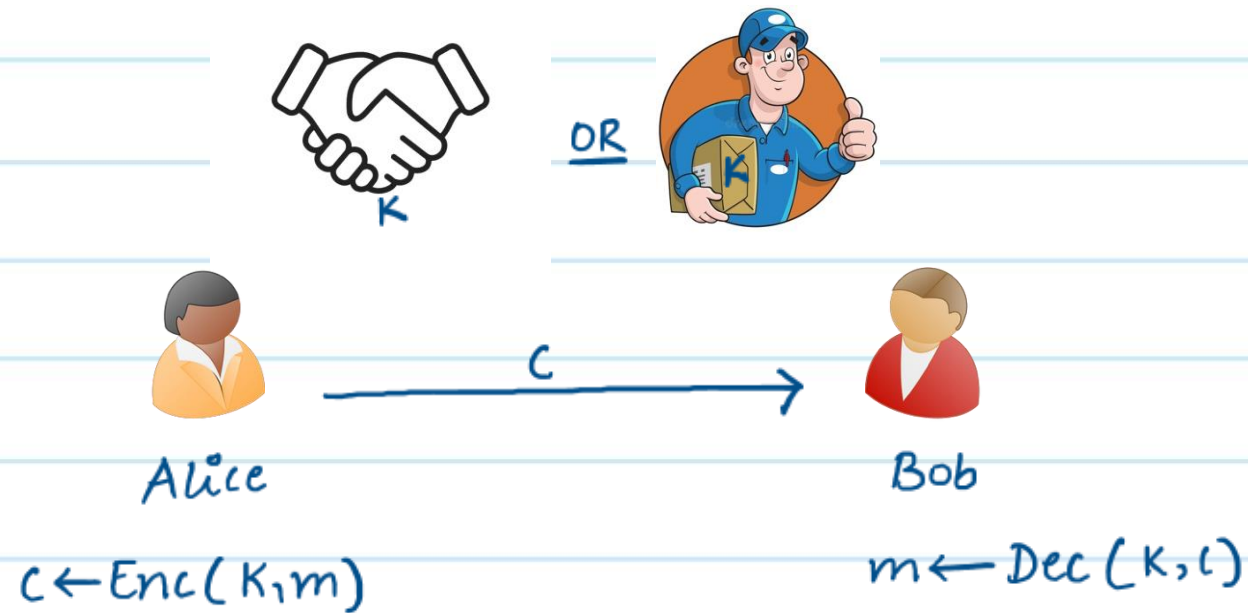
Key-Exchange Problem

- * It is more realistic to believe that all of these are insecure communication channels.
- * How do Alice and Bob share this key over an insecure communication channel?



Key-Exchange Problem: Attempt 1

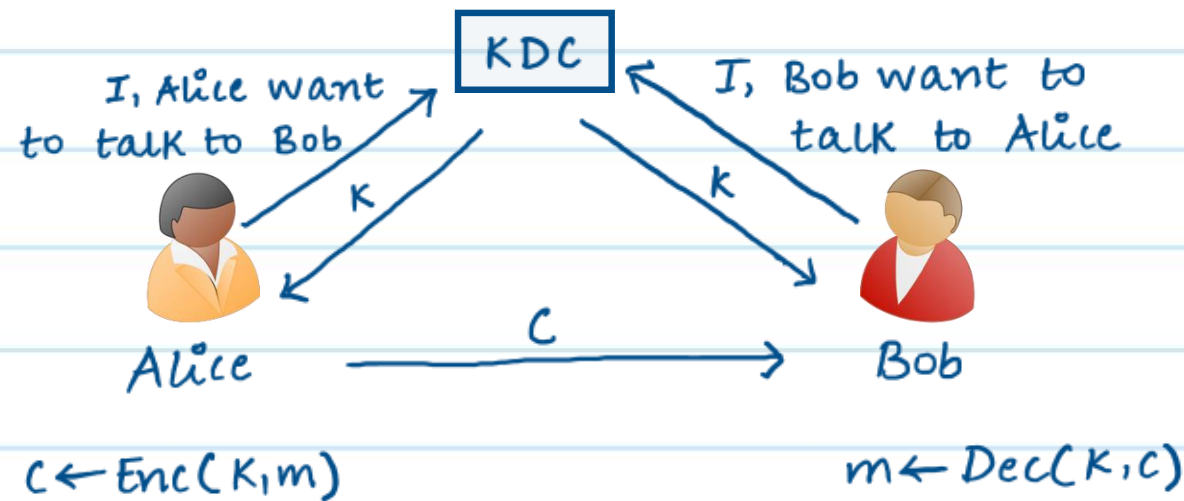
Trusted Courier or face-to-face meeting.



PROBLEM. Not practical for large-scale applications!

Key-Exchange Problem: Attempt 2

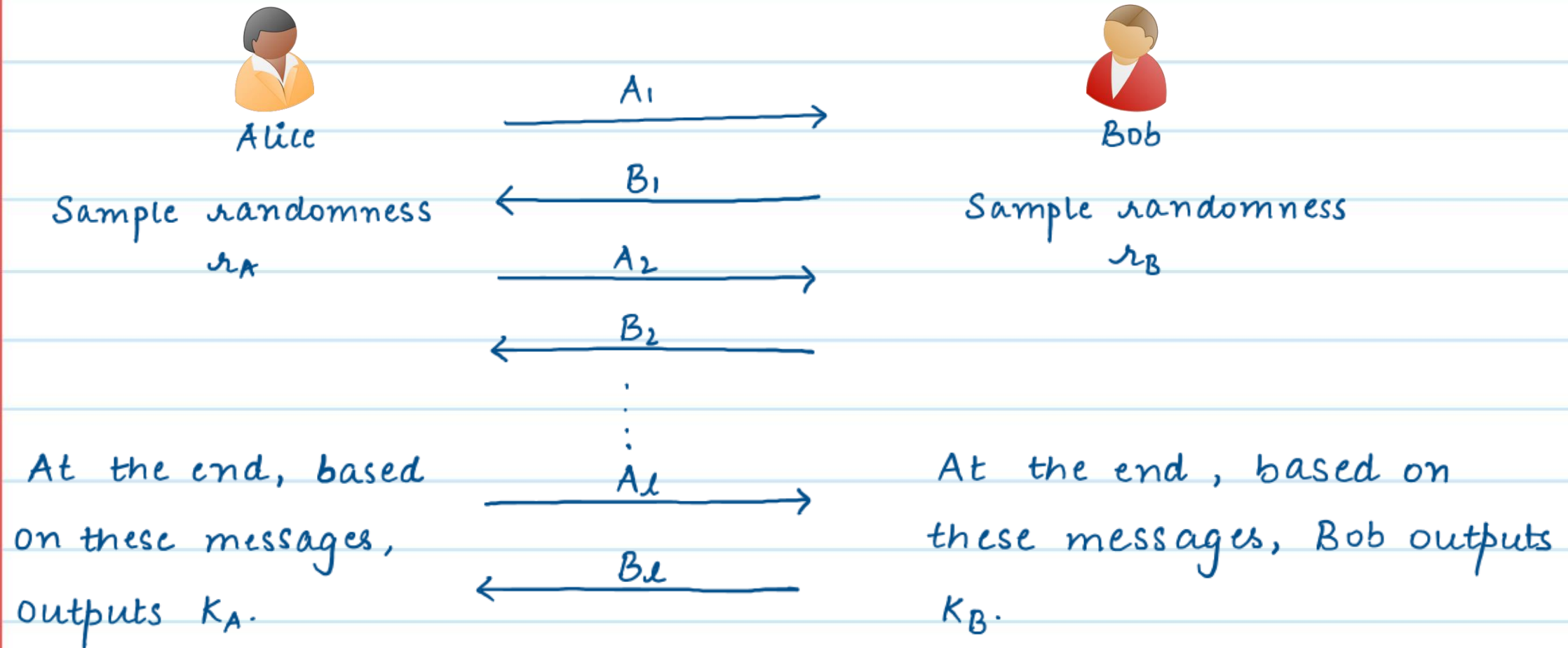
Key-distribution centers: have a trusted center with whom Alice and Bob are *registered* and they can request the trusted center for the shared key.



PROBLEM: Requires trusting these centers, creates a single-point of failure, doesn't scale.

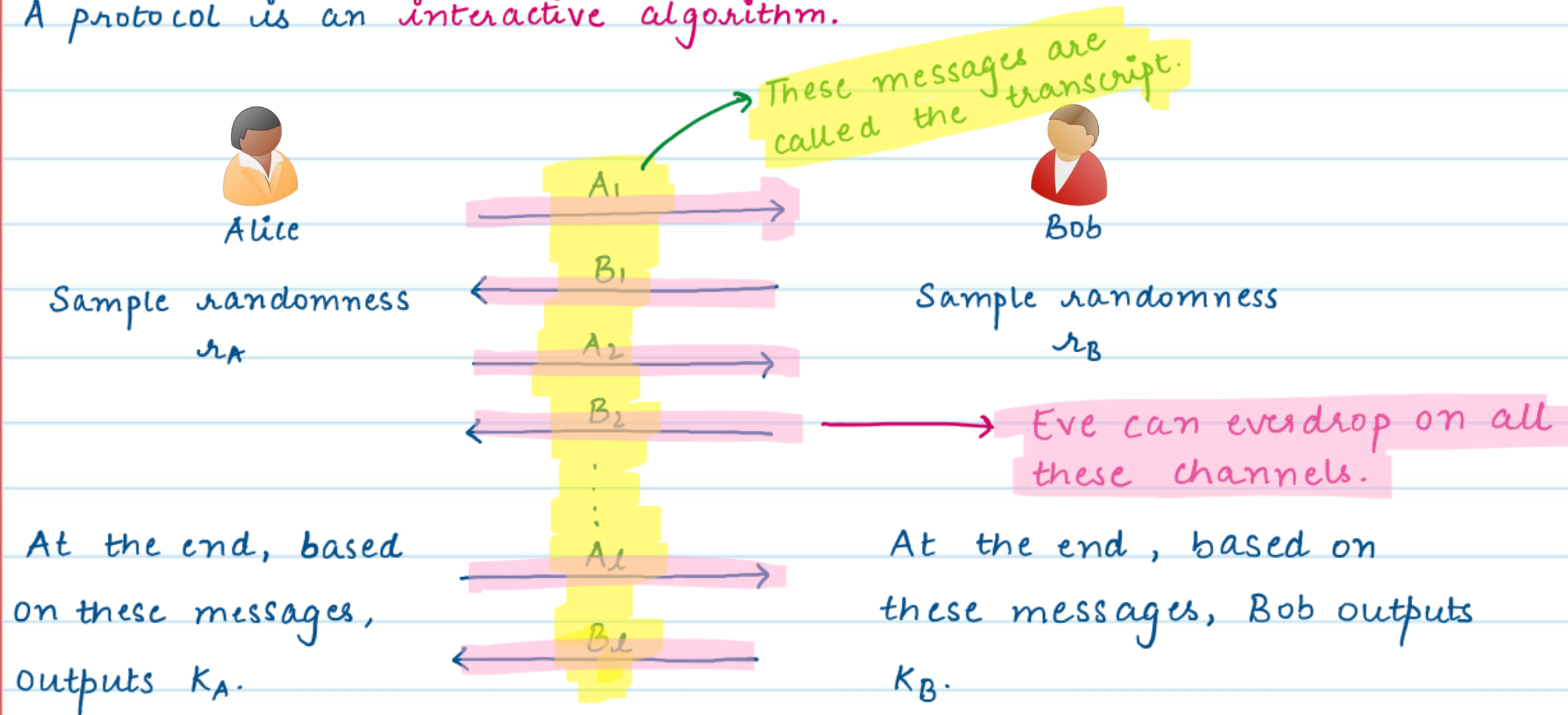
Key-Exchange Protocol

- * A way to establish a secret key via insecure communication channel.
- * A protocol is an *interactive algorithm*.



Key-Exchange Protocol

- * A way to establish a secret key via insecure communication channel.
- * A protocol is an **interactive algorithm**.



Key-Exchange Protocol

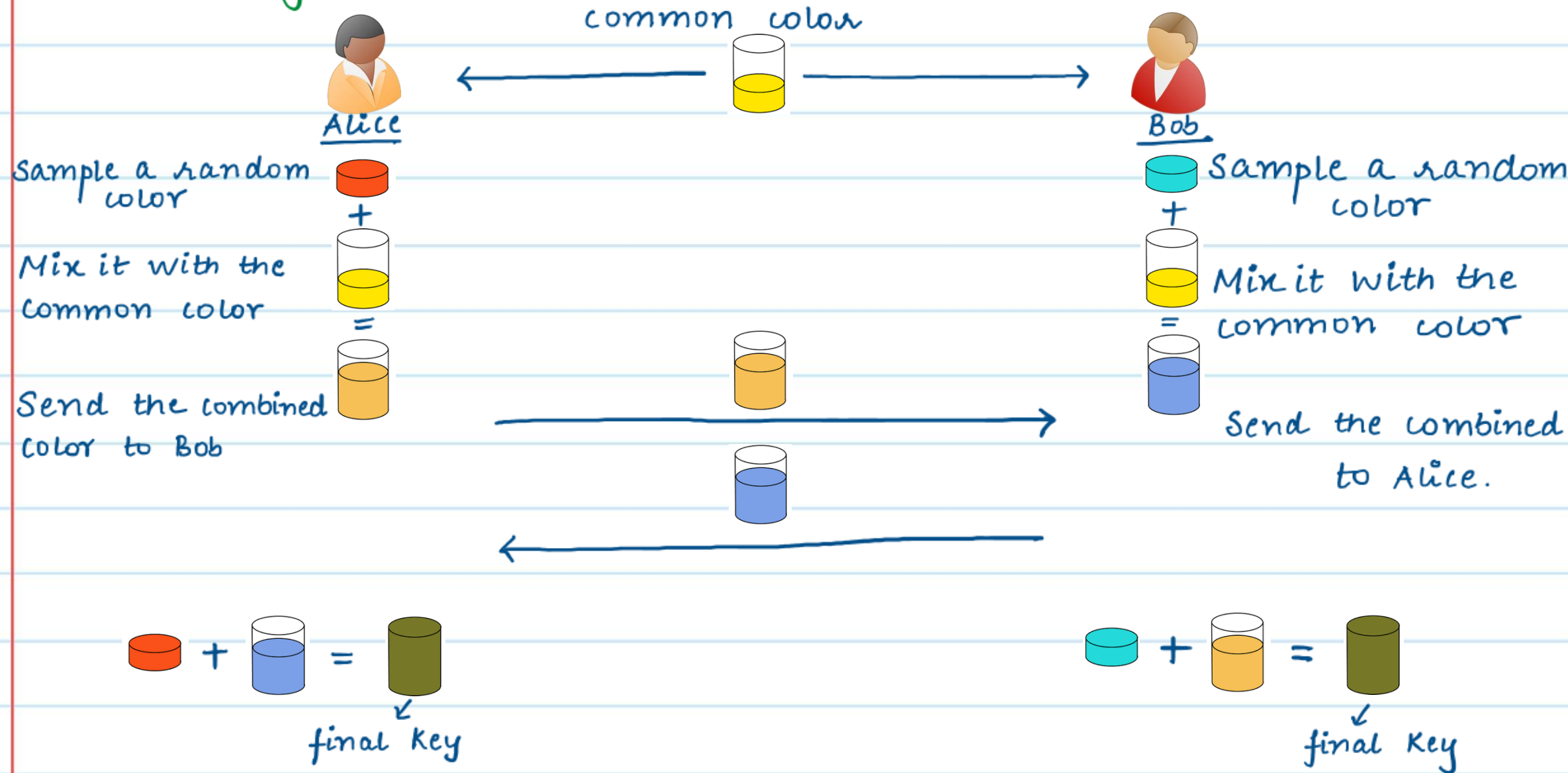
* For correctness, we want:

$\Pr[K_A = K_B] = 1$, for all choices of random values r_A & r_B .

* For privacy, we want:

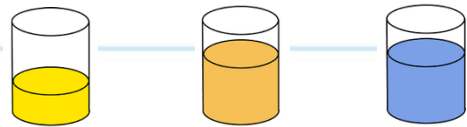
The messages $\{(A_i, B_i)\}_{i \in [n]}$ that Eve can also see, should not reveal any information about the final Key $K_A = K_B$ to Eve.

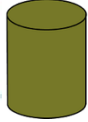
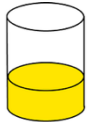
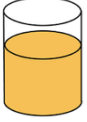
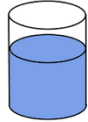
Key-Exchange Protocol



Correctness: Both Alice and Bob mix the same set of colors and hence end up with the same final Key.

Security: What does Eve see in this protocol?



- * In order to learn the final Key , Eve needs to know which color was mixed with  to get  or .
- * However, this color separation is hard.
- * Hence, Eve cannot learn the Key with very high probability.

Basic Group Theory

Recall the definition of groups.

Definition: A group, represented by $(G, \circ)$, is defined by a set G and a binary operator $\circ$ that satisfies the following properties:

* Closure: $\forall a, b \in G$, we have $a \circ b \in G$

* Associativity: $\forall a, b, c \in G$, we have $(a \circ b) \circ c = a \circ (b \circ c)$

* Identity: $\exists$ an element $e \in G$, such that $\forall a \in G$, we have $a \circ e = a$

* Inverse: $\forall$ elements $a \in G$, $\exists$ an element $(-a) \in G$, such that $a \circ (-a) = e$

Basic Group Theory

* Group exponentiation: For a group $(G, \cdot)$ and group elements $g, h \in G$

$$g^m = \underbrace{g \cdot g \cdots g}_m, \quad g^0 = e, \quad g^{-m} = (g^{-1})^m$$
$$g^{m_1} \cdot g^{m_2} = g^{m_1 + m_2}, \quad (g^{m_1})^{m_2} = g^{m_1 \cdot m_2}, \quad g^m \cdot h^m = (g \cdot h)^m$$

* For a finite group, we use $|G|$ to denote its order (# of elements)

* Let G be a group of order q .

* G is a cyclic group if $\exists g \in G$, s.t. $\{g^0, g^1, \dots, g^{q-1}\} = G$.

g is called the generator of G .

Diffie - Hellman Assumptions

Let's sample a cyclic group $(G, \cdot)$ of order q , with generator g .

* **Decisional Diffie-Hellman (DDH) Assumption:** Sample $x, y, z \xleftarrow{\$} \mathbb{Z}_q$, compute $h_1 = g^x$, $h_2 = g^y$. Given (G, q, g, h_1, h_2) , it is computationally hard to distinguish between g^{xy} and g^z .

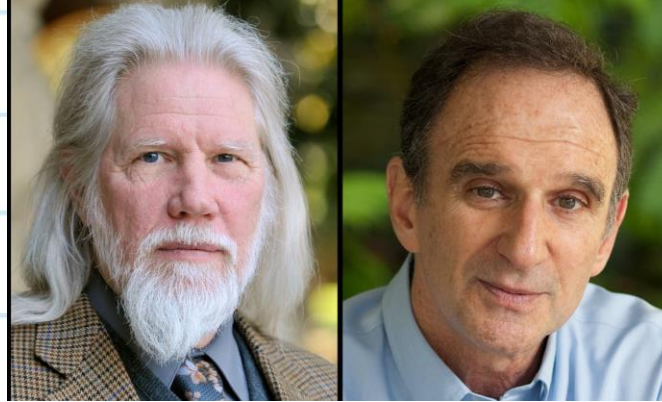
In other words,

$$(G, g, q, h_1, h_2, g^{xy}) \underset{\downarrow}{\approx}_c (G, g, q, h_1, h_2, g^z)$$

computationally
indistinguishable.

* Informally speaking, this also means that g^x, g^y do not leak information about g^{xy} .

Diffie - Hellman Key-Exchange Protocol

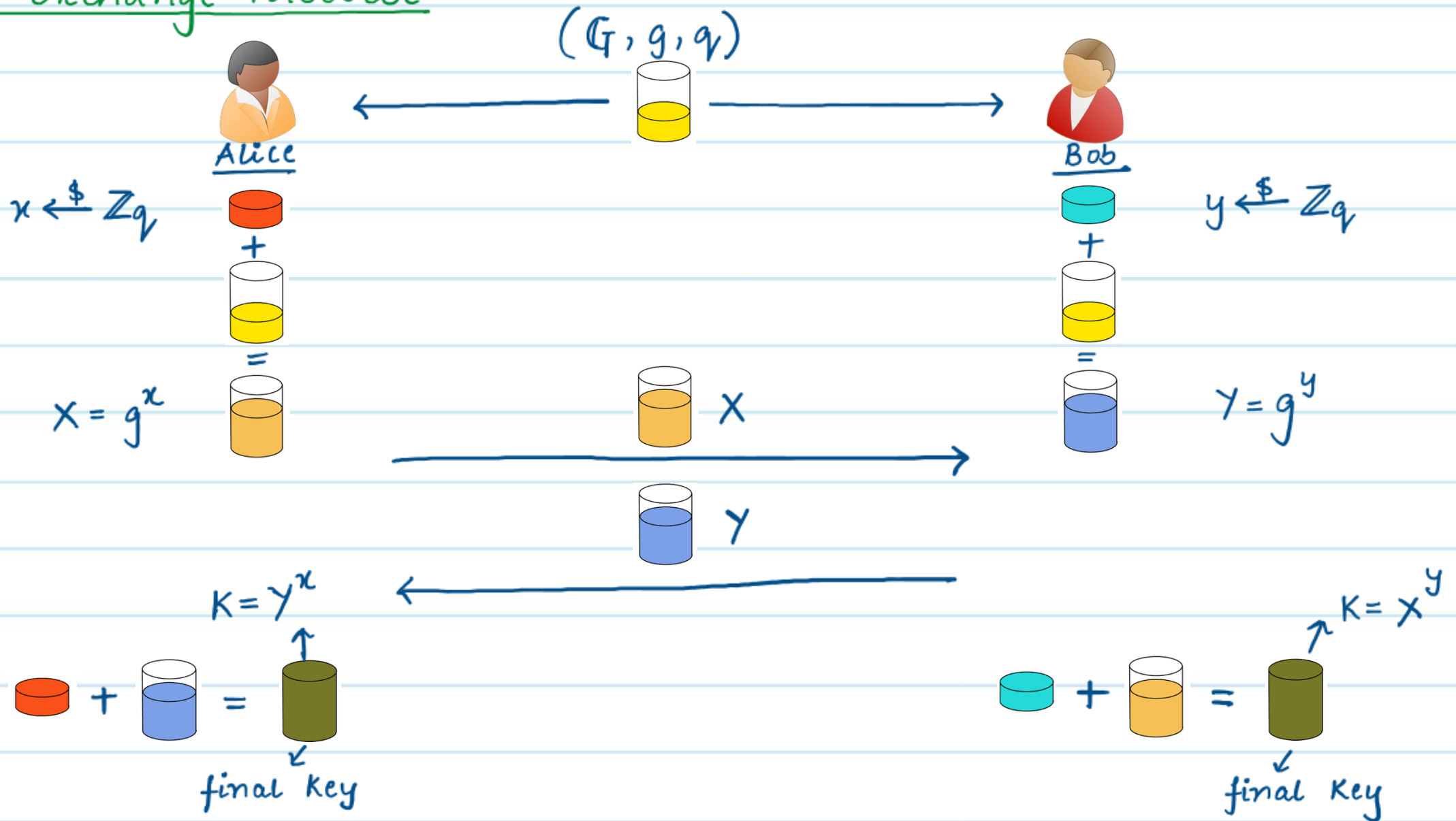


Whitfield
Diffie

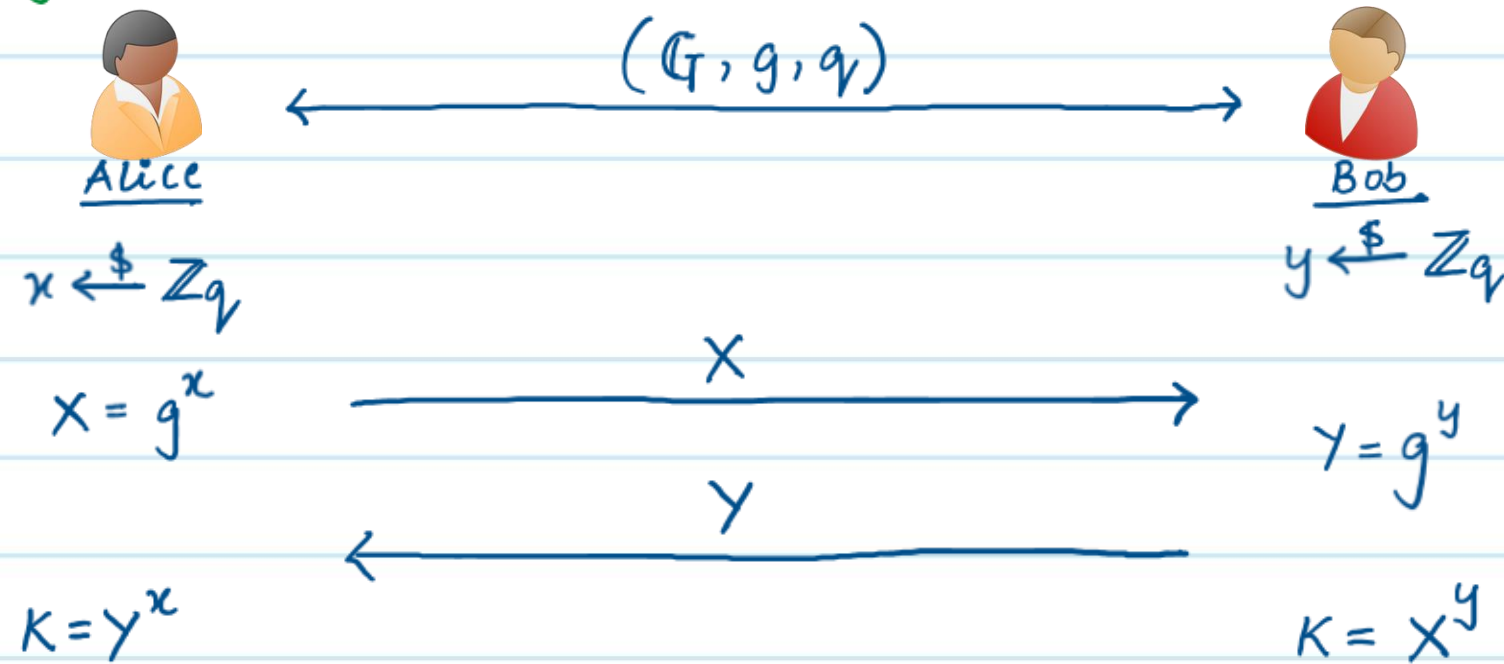
Martin
Hellman

- * A Key exchange protocol from 1976.
- * Security of this protocol is based on the decisional Diffie-Hellman assumption
- * A concrete (mathematical) way to implement the coloring based protocol.

Diffie-Hellman Key-Exchange Protocol



Diffie-Hellman Key-Exchange Protocol



Correctness. $Y^x = (g^y)^x = g^{xy}$
 $X^y = (g^x)^y = g^{xy}$

Both Alice and Bob
compute the same key.

Privacy: Eve only sees (G, g, q) ,
 $X = g^x$ & $Y = g^y$. From the DDH
assumption, we know that this
does not leak information about
the key $K = g^{xy}$.